1. (A) From Section 5.2 in M4D, given a budget of t hash functions and a threshold τ, the value of r is approximately given by:

$$r \approx -\log_{\tau}(t) = -\log_{.75}(200) \approx 19$$

Furthermore, the number of hash functions t is split into r and b; therefore, b—the number of rows within each band—is the ratio between t and r, which corresponds to:

$$b = t/r = 200/19 \approx 11$$

We can verify that these values yield a good separation around τ by plugging them into $f(s) = 1 - (1 - s^b)^r$ and check for an S-shaped curve with steepest slope/rise at $\tau = 0.75$.

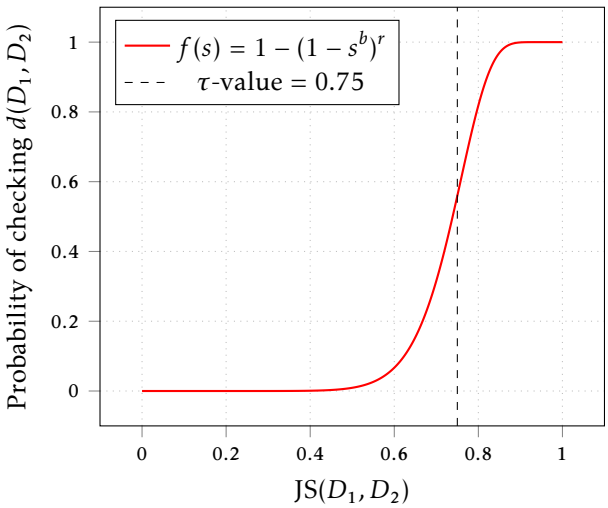

Figure 1. Probability that the distance $d(D_1, D_2)$ is checked in a LSH scheme with $b = 11$ bands with $r = 19$ hashes each, as function of $s = \mathrm{JS}(D_1, D_2)$. See Listing 1.

(B) The six numbers outlining the probability for each pair of the four objects being estimated as similar can be retrieved as follows. First, we note that

$$\mathrm{JS}(A,B) = 0.77 \quad \mathrm{JS}(A,C) = 0.25 \quad \mathrm{JS}(A,D) = 0.33$$
$$\mathrm{JS}(B,C) = 0.20 \quad \mathrm{JS}(B,D) = 0.55 \quad \mathrm{JS}(C,D) = 0.91$$

and each one of these values correspond to s. Furthermore, for a threshold $\tau = 0.75$, we want $f(s)$ parameterized with $r = 19$ and $b = 11$. Hence plugging each s into $f(s)$, we obtain the following results:

	B	C	D
A	0.66823	0.00000	0.00010
B	—	0.00000	0.02614
C	—	—	0.99975

Table 1. Probability that each pair of documents will have similarity $> \tau$.

2. (A) From Section 4.6.4 in M4D, we construct a single random unit vector $\boldsymbol{x} \in \mathbb{R}^{10}$ (chosen uniformly over the space of all unit vectors) as follows: (i) sample $u_1, u_2 \overset{\text{i.i.d.}}{\sim} \mathcal{U}(0,1)$, (ii) define 1-dimensional gaussian random variable $\mathcal{Y} = \sqrt{-2\ln(u_1)}\cos(2\pi u_2)$, (iii) sample each $x_1, \ldots, x_{10} \overset{\text{i.i.d.}}{\sim} \mathcal{Y}$, and (iv) scale $\boldsymbol{x}$ by the inverse of its norm to obtain the desired random unit vector $\tilde{\boldsymbol{x}} = \boldsymbol{x}/\|\boldsymbol{x}\|_2$.

(B) The pairwise dot product CDF is illustrated below.

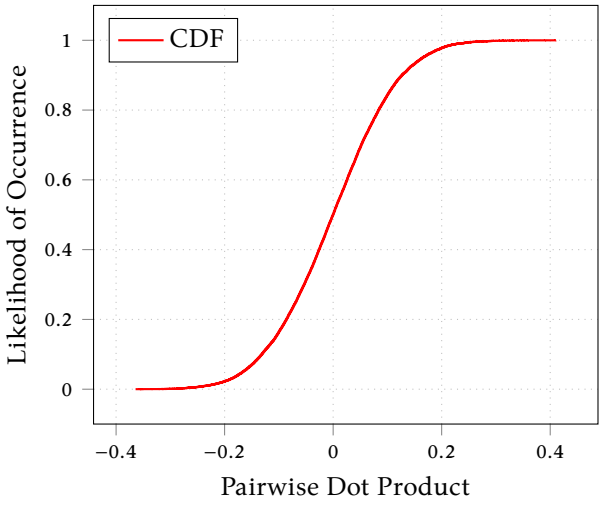

Figure 2. Dot product for each pair of random unit vectors in $\mathbb{R}^{100}$. See Listing 2.

3. (A) The pairwise angular similarity CDF is illustrated below. The number of similarities that exceed $\tau = 0.75$ is **107004**.

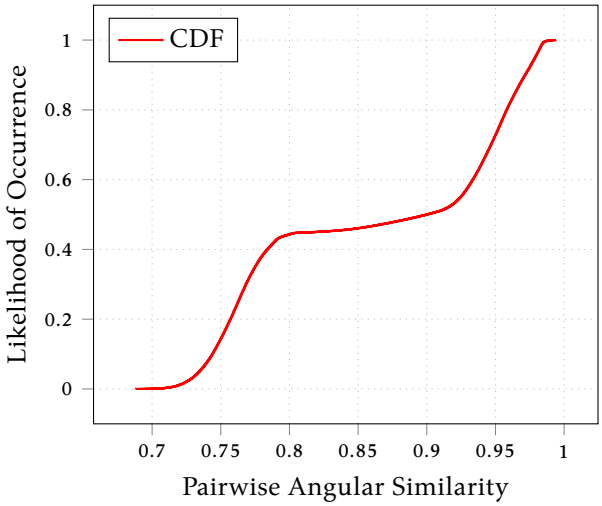

Figure 3. Angular similarities of pairwise unit vectors from `R.txt` dataset. See Listing 3.

(B) The pairwise angular similarity CDF is illustrated below. The number of similarities that exceed $\tau = 0.75$ is **0**.

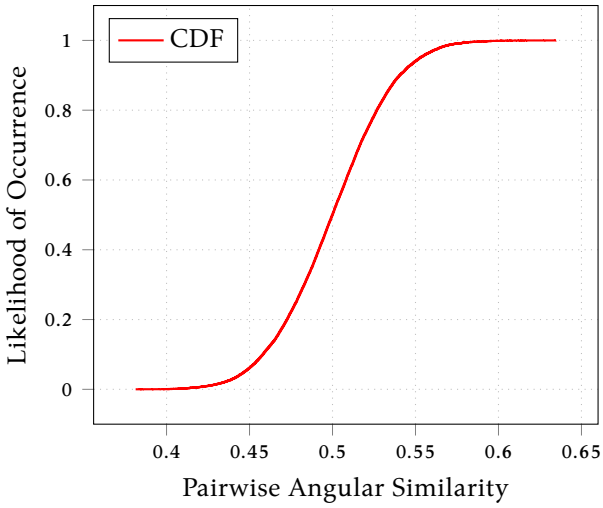

Figure 4. Angular similarities for each pair of random unit vectors in $\mathbb{R}^{100}$. See Listing 2.

Appendix

```python
import numpy as np

B_VALUE = 11
R_VALUE = 19

class F:
    def __init__(self, b: int, r: int) -> None:
        self.b = b
        self.r = r

    def apply(self, s: np.ndarray) -> np.ndarray:
        return 1 - np.power(1 - np.power(s, self.b), self.r)

def main() -> int:
    s_values = np.linspace(0.0, 1.0, 1000)
    f_values = F(B_VALUE, R_VALUE).apply(s_values)
    np.savetxt('data/lsh-values.csv', np.column_stack((s_values, f_values)), delimiter=',')
    return 0

if __name__ == '__main__':
    raise SystemExit(main())
```

Listing 1. lsh-script.py

```python
import numpy as np

from itertools import combinations, starmap

TAU_VAL = .75
D_VALUE = 100
T_VALUE = 200

def random_uniform(lowerbound: float, upperbound: float, size: tuple) -> np.ndarray:
    return np.random.uniform(lowerbound, upperbound, size)

def random_normal(lowerbound: float, upperbound: float, size: tuple) -> np.ndarray:
    u_one = random_uniform(lowerbound, upperbound, size)
    u_two = random_uniform(lowerbound, upperbound, size)
    return np.sqrt(-2 * np.log(u_one)) * np.cos(2 * np.pi * u_two)

def normalize(v: np.ndarray) -> np.ndarray:
    v_norm = np.linalg.norm(v)
    return v / v_norm

def angular_similarity(u: np.ndarray, v: np.ndarray) -> float:
    return 1 - np.arccos(np.dot(u, v)) / np.pi

def main() -> int:
    vectors = random_normal(0.0, 1.0, (T_VALUE, D_VALUE))
    vectors = np.array(list(map(normalize, vectors)))
    combins = list(combinations(vectors, 2))

    dot_products = np.array(list(starmap(np.dot, combins)))
    angular_sims = np.array(list(starmap(angular_similarity, combins)))

    np.savetxt('data/dot-products.csv', dot_products, delimiter=',')
    np.savetxt('data/angular-sims.csv', angular_sims, delimiter=',')

    print((angular_sims > TAU_VAL).sum())
    return 0

if __name__ == '__main__':
    raise SystemExit(main())
```

Listing 2. gen-unit-vec.py

```python
import numpy as np

from itertools import combinations, starmap

TAU_VAL = .75
DATASET = 'data/R.txt'

def normalize(v: np.ndarray) -> np.ndarray:
    v_norm = np.linalg.norm(v)
    return v / v_norm

def angular_similarity(u: np.ndarray, v: np.ndarray) -> float:
    return 1 - np.arccos(np.dot(u, v)) / np.pi

def load_dataset(dataset: str) -> np.ndarray:
    vectors = np.loadtxt(dataset)
    return vectors

def main() -> int:
    vectors = np.array(list(map(normalize, load_dataset(DATASET))))
    combins = combinations(vectors, 2)

    angular_sims = np.array(list(starmap(angular_similarity, combins)))
    np.savetxt('data/angular-sims-r-dataset.csv', angular_sims, delimiter=',')

    print((angular_sims > TAU_VAL).sum())
    return 0

if __name__ == '__main__':
    raise SystemExit(main())
```

Listing 3. angular-sim.py